HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO THỰC HÀNH
DỊCH NGƯỢC
AN TOÀN GAME
BENTOLOOP

Giảng viên hướng dẫn: Cao Minh Thắng
Lớp: D24CQGA02-B

Sinh viên **MSV**
Nguyễn Thành Vinh B24DCGA162

HÀ NỘI - 10/2026

MỤC LỤC

1 Mục tiêu và kết quả thực hành 2
2 Decompile APK và lấy thư viện native 2
3 Xác định hàm kiểm tra license từ dữ liệu dump 4
4 Import thư viện và gắn nhãn trong Ghidra 5
4.1 Import libil2cpp và kiểm tra Image Base 5
4.2 Gắn tên từ script json 6
5 Phân tích điều kiện chấp nhận license 7
5.1 Theo dõi đầu vào trong VerifyLicense 7
5.2 Xác định nhánh trả về thành công 8
6 Tìm key và kiểm chứng trên APK gốc 8
6.1 Đọc chuỗi trong Ghidra và đối chiếu JSON 8
6.2 Thử key trên bản APK gốc 9
7 Sửa VerifyLicense và xuất thư viện 10
7.1 Sửa hai lệnh đầu hàm 10
7.2 Xuất file có chứa phần mã đã sửa 11
7.3 Thay thư viện trong thư mục đã Decompile 11
8 Build APK và kiểm thử bản đã sửa 12
8.1 Cập nhật công cụ build 12
8.2 Làm mới framework và Compile 13
8.3 Lấy APK đã build và ký 14
8.4 Kiểm thử với chuỗi khác key đã tìm được 15
9 Nhận xét và phương án phòng tránh 15
9.1 Nguyên nhân cơ chế có thể bị vòng tránh 15
9.2 Các biện pháp giảm rủi ro 16
9.3 Kết luận thực hành 16
Tài liệu tham khảo 16

1 Mục tiêu và kết quả thực hành

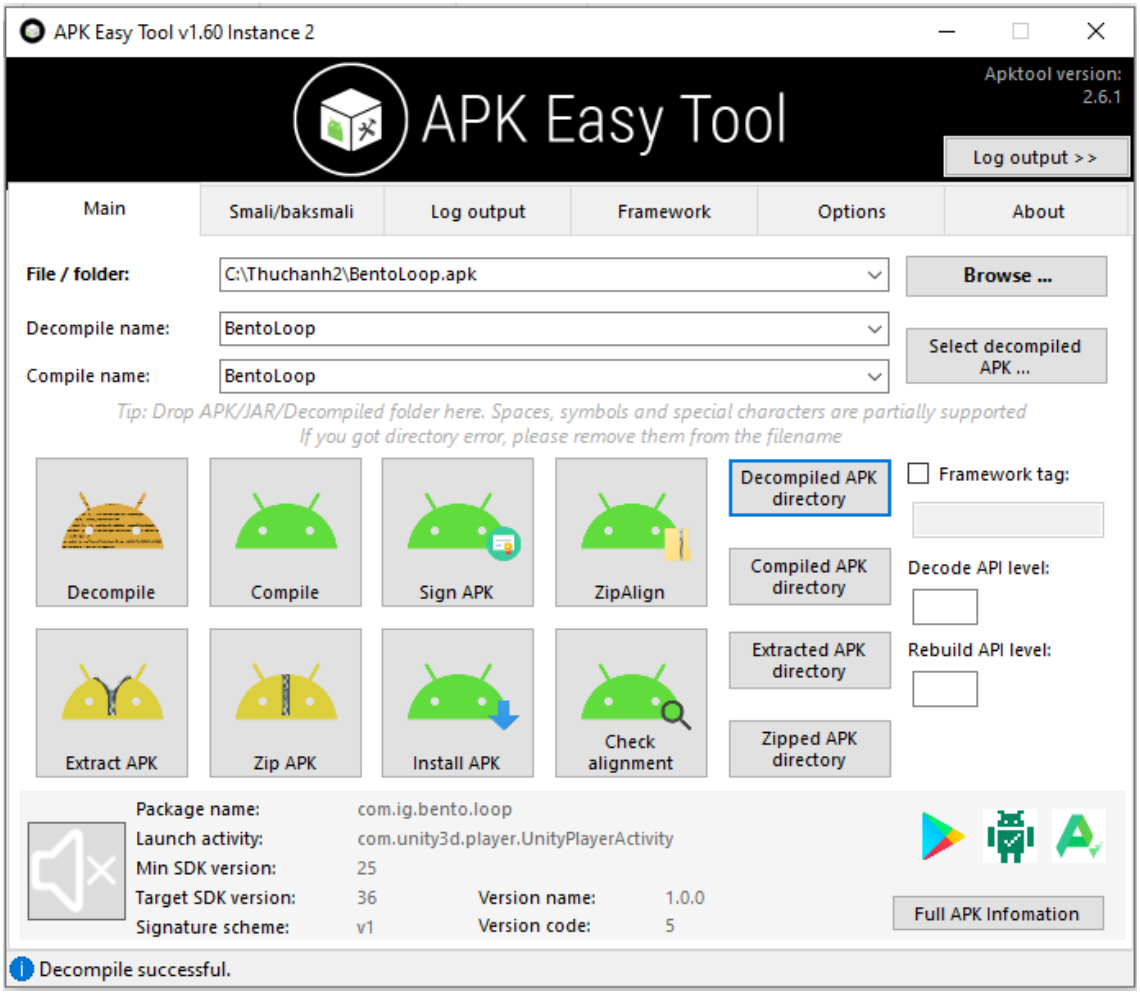
Bài thực hành phân tích cơ chế kiểm tra license của BentoLoop, tìm key được chấp nhận và sửa thư viện native để build lại APK. Kết quả tìm được là BENTO-EASY-2025. APK gốc báo kích hoạt thành công khi nhập key này và chuyển vào Level 1. Sau khi sửa VerifyLicense để trả về thành công ngay đầu hàm, APK được build và ký lại bằng APK Easy Tool. Bản APK mới báo kích hoạt thành công với chuỗi vinh123.

Quá trình thực hiện gồm giải mã cấu trúc APK, tra cứu dữ liệu IL2CPP, phân tích mã native trong Ghidra, sửa hai lệnh ARM64 và kiểm thử ứng dụng. Các ảnh minh chứng được đặt cùng phần giải thích tương ứng để theo dõi dữ liệu đầu vào, thao tác và kết quả của từng bước.

Thành phần	Thông tin sử dụng
Môi trường	Windows trong máy ảo VMware Workstation; kiểm thử trên thiết bị Android kết nối với máy thật
Ứng dụng	BentoLoop.apk; package com.ig.bento.loop; version name 1.0.0; version code 5
Thư viện phân tích	lib/arm64-v8a/libil2cpp.so; kiến trúc AARCH64, little-endian, 64 bit
APK Easy Tool	Phiên bản 1.60; Decompile bằng Apktool 2.6.1; Compile bằng Apktool 2.12.1
Ghidra	Phiên bản 12.1.2; project Thuchanh2; Image Base 0x100000
Dữ liệu dump	Gói libil2cpp_il2cppdump_20261007_0259 được cung cấp trong bài thực hành

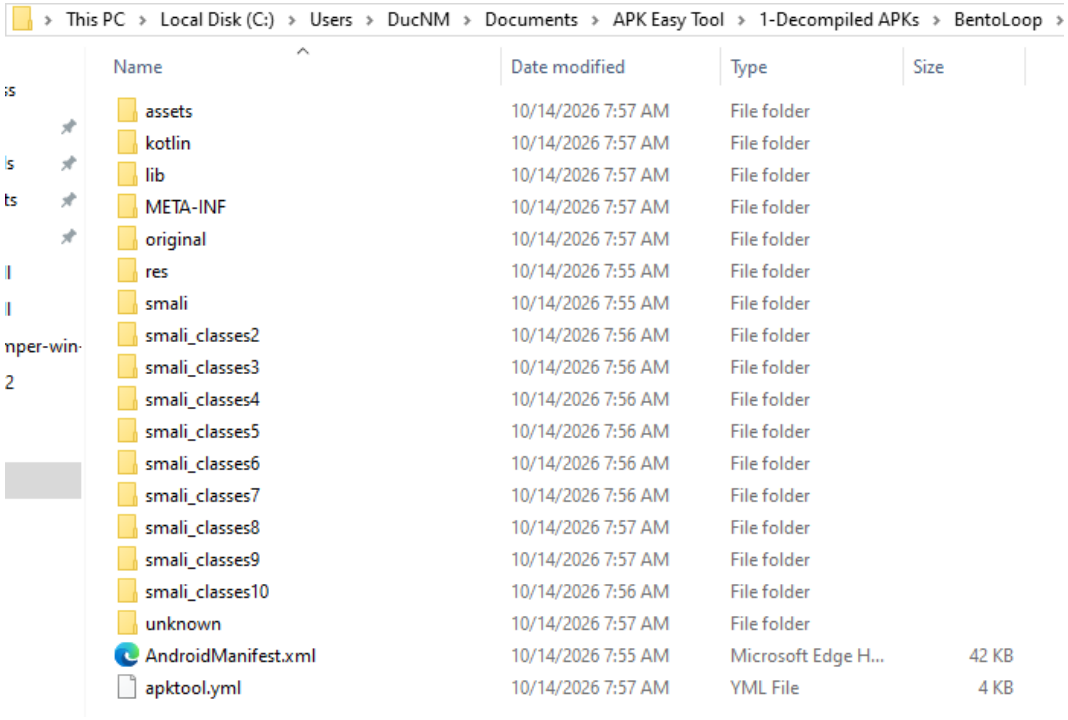
2 Decompile APK và lấy thư viện native

File đầu vào là C:\Thuchanh2\BentoLoop.apk trong máy ảo. Tại APK Easy Tool, Browse được dùng để chọn file, sau đó bấm Decompile. Công cụ hiển thị Decompile successful và nhận dạng package com.ig.bento.loop. APK khai báo min SDK 25 và target SDK 36.



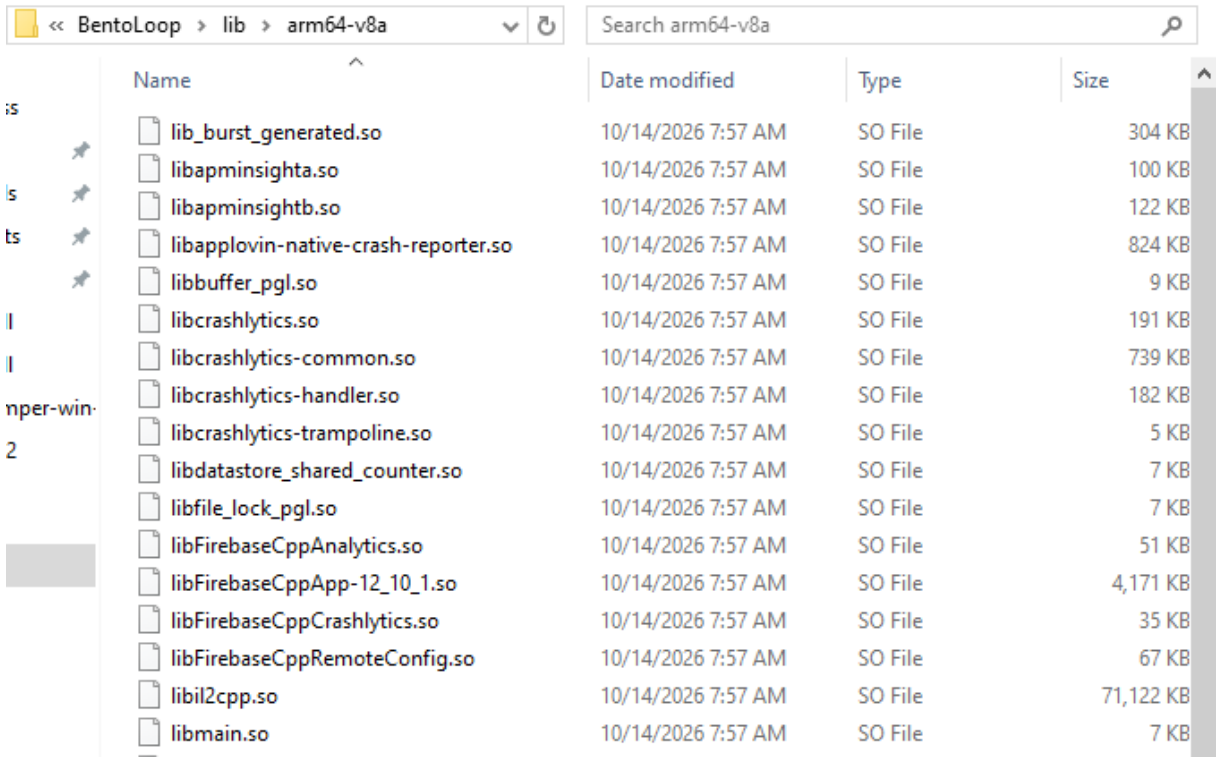
Hình 1. Decompile BentoLoop.apk bằng APK Easy Tool

Kết quả nằm trong thư mục BentoLoop dưới 1-Decompiled APKs. Thư mục có assets, lib, res, các thư mục smali, AndroidManifest.xml và apktool.yml. File apktool.yml giúp công cụ nhận diện thư mục này khi Compile lại.



Hình 2. Cấu trúc thư mục BentoLoop sau Decompile

Thư viện được chọn để phân tích là libil2cpp.so trong lib\arm64-v8a. Đây là mã native của phần IL2CPP trong ứng dụng. File có dung lượng hiển thị 71.122 KB, tương ứng 72.828.928 byte. Ở bước sửa hàm, chương trình được lưu thành bản sao trong Ghidra để giữ phần phân tích của bản gốc.



Hình 3. Thư viện libil2cpp.so của bản ARM64

```
Thư mục làm việc
C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop

Thư viện cần thay khi build
lib\arm64-v8a\libil2cpp.so
```

3 Xác định hàm kiểm tra license từ dữ liệu dump

Bộ dump được cung cấp gồm dump.cs, script.json, stringliteral.json, các script hỗ trợ Ghidra và thư mục DummyDll. Trong bước này, dữ liệu dump được dùng để tra cứu tên lớp và địa chỉ; thư viện native được lấy từ APK đã Decompile.

Tìm LicenseVerifier trong dump.cs cho kết quả lớp static thuộc namespace GeckoArrow.Runtime.LoadingSystem. VerifyLicense nhận string inputKey và trả về bool. Lớp còn có CheckTransformedLicense, IsLicenseActivated, SaveLicenseActivated và field EncryptedBytes. Thân hàm trong dump.cs không chứa logic thực thi nên cần mở thư viện trong Ghidra để đọc mã native.

```
// Namespace: GeckoArrow.Runtime.LoadingSystem
public static class LicenseVerifier // TypeDefIndex: 12855
{
        // Fields
        private static readonly byte[] EncryptedBytes; // 0x0

        // Methods

        // RVA: 0x1E76FA0 Offset: 0x1E72FA0 VA: 0x1E76FA0
        public static bool CheckTransformedLicense(string key) { }

        // RVA: 0x1E7689C Offset: 0x1E7289C VA: 0x1E7689C
        public static bool VerifyLicense(string inputKey) { }

        // RVA: 0x1E76EF4 Offset: 0x1E72EF4 VA: 0x1E76EF4
        public static bool IsLicenseActivated() { }

        // RVA: 0x1E769FC Offset: 0x1E729FC VA: 0x1E769FC
        public static void SaveLicenseActivated(bool activated = True) { }

        // RVA: 0x1E77110 Offset: 0x1E73110 VA: 0x1E77110
        private static void .cctor() { }
}
```

Hình 4. Lớp LicenseVerifier và địa chỉ các phương thức trong dump.cs

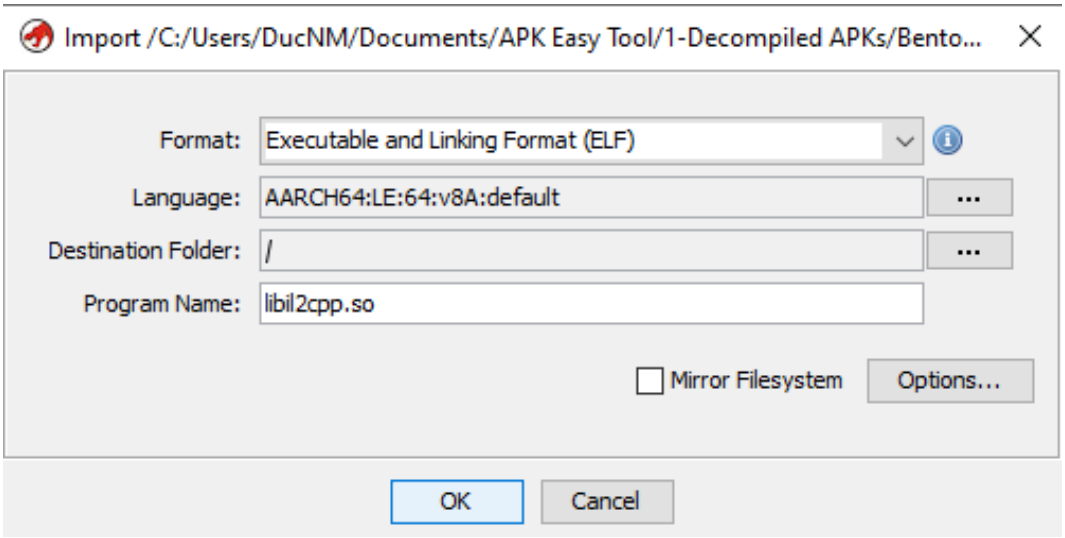
Phương thức	RVA	Địa chỉ trong Ghidra
VerifyLicense	0x1E7689C	0x1F7689C
CheckTransformedLicense	0x1E76FA0	0x1F76FA0
IsLicenseActivated	0x1E76EF4	0x1F76EF4
SaveLicenseActivated	0x1E769FC	0x1F769FC
.cctor	0x1E77110	0x1F77110

RVA là địa chỉ tương đối so với Image Base. Cột địa chỉ Ghidra trong bảng đã cộng Image Base 0x100000. Với VerifyLicense, dump còn ghi File Offset 0x1E7289C. Offset này chỉ vị trí trong file và khác địa chỉ hiển thị trong Ghidra.

4 Import thư viện và gắn nhãn trong Ghidra

4.1 Import libil2cpp và kiểm tra Image Base

libil2cpp.so được import vào project Thuchanh2. Ghidra nhận dạng file ELF với language AARCH64:LE:64:v8A:default. Sau khi mở CodeBrowser, phân tích tự động được chạy để nhận diện các hàm và tham chiếu.



Hình 5. Ghidra nhận dạng thư viện ELF ARM64

Memory Map hiển thị Image Base 00100000. Vì vậy, địa chỉ cần nhập trong Go To được tính bằng RVA cộng 0x100000. Phép quy đổi này được dùng nhất quán khi tìm hàm và đối chiếu chuỗi.

```
Địa chỉ Ghidra = Image Base + RVA
0x00100000 + 0x01E7689C = 0x01F7689C
```

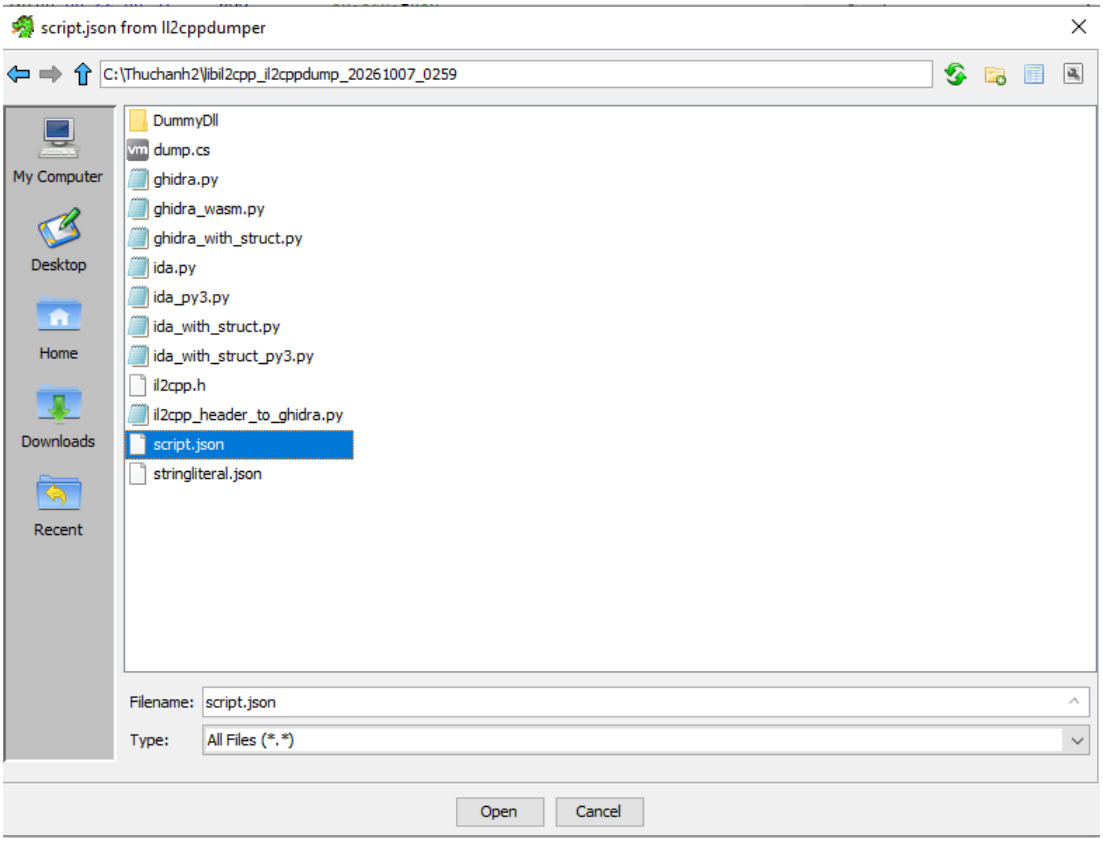
Memory Map - Image Base: 00100000

Name	Start	End	Length	R	W	X	Volatile	Artificial	Overlayed
segment_1.1	00100000	0010026f	0x270	☑	☐	☐	☐	☐	
.note.android.ident	00100270	00100307	0x98	☑	☐	☐	☐	☐	
.note.gnu.build-id	00100308	0010032b	0x24	☑	☐	☐	☐	☐	
.dynsym	00100330	0011001f	0xfcf0	☑	☐	☐	☐	☐	
.gnu.version	00110020	00111533	0x1514	☑	☐	☐	☐	☐	
.gnu.version_r	00111534	00111593	0x60	☑	☐	☐	☐	☐	
.gnu.hash	00111598	001154d7	0x3f40	☑	☐	☐	☐	☐	
.dynstr	001154d8	00131422	0x1bf4b	☑	☐	☐	☐	☐	
.rela.dyn	00131428	00b9910f	0xa67ce8	☑	☐	☐	☐	☐	
.rela.plt	00b99110	00b9c877	0x3768	☑	☐	☐	☐	☐	
.gcc_except_table	00b9c878	00c47fab	0xab734	☑	☐	☐	☐	☐	
.rodata	00c47fb0	010d2a4b	0x48aa9c	☑	☐	☐	☐	☐	
.eh_frame_hdr	010d2a4c	012c07b7	0x1edd6c	☑	☐	☐	☐	☐	
.eh_frame	012c07b8	01b22b8f	0x8623d8	☑	☐	☐	☐	☐	
.text	01b26b90	01e96f6b	0x3703dc	☑	☐	☑	☐	☐	
il2cpp	01e96f6c	04181d67	0x22eadfc	☑	☐	☑	☐	☐	
.plt	04181d70	0418427f	0x2510	☑	☐	☑	☐	☐	

Hình 6. Memory Map xác nhận Image Base 0x100000

4.2 Gắn tên từ script json

Thư mục chứa ghidra.py được thêm vào Script Manager. Script được chạy với file script.json trong gói dump BentoLoop. Kết quả gắn tên giúp Decompiler hiển thị các phương thức như LicenseVerifier$$VerifyLicense, String.Trim và String.Equals thay cho phần lớn tên tự động.



Hình 7. Chọn script.json của BentoLoop để gắn tên trong Ghidra

FUN_01f7689c là tên tự động của Ghidra tại địa chỉ 0x1F7689C. Sau khi áp dụng dữ liệu dump, cùng vị trí đó có tên GeckoArrow.Runtime.LoadingSystem.LicenseVerifier$$VerifyLicense. Nhãn được đổi để dễ đọc mã giả; mã máy chỉ thay đổi ở bước patch sau đó.

5 Phân tích điều kiện chấp nhận license

5.1 Theo dõi đầu vào trong VerifyLicense

Go To được dùng để mở VerifyLicense tại 0x1F7689C. Phần đầu hàm gọi String.IsNullOrWhiteSpace để kiểm tra đầu vào. Với chuỗi có nội dung, String.Trim bỏ khoảng trắng ở hai đầu, sau đó String.Equals so sánh kết quả với StringLiteral_2770. Tham số kiểu so sánh hiển thị trong mã giả là 4.

```
}
uVar2 = System.String$$IsNullOrWhiteSpace(param_1,0);
if ((uVar2 & 1) == 0) {
  if (param_1 == 0) {
                    /* WARNING: Subroutine does not return */
    FUN_01da5340();
  }
  uVar3 = System.String$$Trim(param_1,0);
  uVar2 = System.String$$Equals(uVar3,StringLiteral_2770,4,0);
  if ((uVar2 & 1) == 0) {
    if (*(int *)(GeckoArrow.Runtime.LoadingSystem.LicenseVerifier_TypeInfo + 0xe4) == 0) {
      thunk_FUN_01e01e10();
    }
    uVar2 = GeckoArrow.Runtime.LoadingSystem.LicenseVerifier$$CheckTransformedLicense(uVar3);
    if ((uVar2 & 1) == 0) {
      uVar2 = GeckoArrow.Runtime.LoadingSystem.SecurityAuthService$$DispatchVerification(uVar3);
      if ((uVar2 & 1) == 0) goto LAB_01f76914;
      iVar1 = *(int *)(UnityEngine.Debug_TypeInfo + 0xe4);
```

Hình 8. VerifyLicense kiểm tra đầu vào và so sánh với StringLiteral_2770

Khi phép so sánh trực tiếp không khớp, luồng gọi CheckTransformedLicense. Nếu nhánh này cũng không chấp nhận chuỗi, SecurityAuthService.DispatchVerification được gọi tiếp. Do đó, StringLiteral_2770 là đầu mối để tìm một key có thể được chấp nhận ngay ở nhánh đầu.

5.2 Xác định nhánh trả về thành công

Trong phần cuối VerifyLicense, nhánh chuỗi khớp chọn thông báo log rồi đi đến phép gán giá trị trả về bằng 1. Nhánh thất bại chuyển đến LAB_01f76914 và trả về 0. Kết hợp phép so sánh và giá trị trả về cho thấy chuỗi tại StringLiteral_2770 có vai trò là key được chấp nhận.

```
        iVar1 = *(int *)(UnityEngine.Debug_TypeInfo + 0xe4);
        puVar4 = &StringLiteral_11021;
      }
    }
    else {
      iVar1 = *(int *)(UnityEngine.Debug_TypeInfo + 0xe4);
      puVar4 = &StringLiteral_11020;
    }
    if (iVar1 == 0) {
      thunk_FUN_01e01e10();
    }
    UnityEngine.Debug$$Log(*puVar4,0);
    uVar3 = 1;
  }
  else {
LAB_01f76914:
    uVar3 = 0;
  }
  return uVar3;
}
```

Hình 9. Nhánh thành công trả về 1 và nhánh thất bại trả về 0

Có thể rút gọn điều kiện quan sát được thành mã giả dưới đây. Phần khởi tạo kiểu dữ liệu và các lời gọi ghi log được lược đi để tập trung vào quyết định chấp nhận license.

```
if (IsNullOrWhiteSpace(inputKey)) return false;
key = Trim(inputKey);
if (Equals(key, StringLiteral_2770, 4)) return true;
if (CheckTransformedLicense(key)) return true;
return SecurityAuthService.DispatchVerification(key);
```

Nhánh CheckTransformedLicense còn đối chiếu từng ký tự với một mảng byte bằng XOR 0x53 và ((i × 13) AND 0xFF). Nhánh so sánh trực tiếp đã cho phép tìm được key dùng trong thử nghiệm, nên không cần giải toàn bộ mảng để hoàn thành kết quả này.

6 Tìm key và kiểm chứng trên APK gốc

6.1 Đọc chuỗi trong Ghidra và đối chiếu JSON

Bấm đúp StringLiteral_2770 trong lời gọi String.Equals chuyển Listing đến 044fce70. Chú thích tại nhãn hiển thị BENTO-EASY-2025. Địa chỉ này tương ứng 0x044FCE70; Ghidra hiển thị chữ số hex thường và không thêm tiền tố 0x.

```
                    StringLiteral_2770                              XREF[3]:     GeckoArrow.Runtime.LoadingSystem...
                                                                                 GeckoArrow.Runtime.LoadingSystem...
                                                                                 043e6878(*)
044fce70 a5 15 00        undefined8  00000000A00015A5h                              BENTO-EASY-2025
         a0 00 00
         00 00
```

Hình 10. StringLiteral_2770 tại 0x044FCE70 chứa BENTO-EASY-2025

Trong stringliteral.json, tìm 43FCE70 cho mục có value là BENTO-EASY-2025 và address là 0x43FCE70. Cộng Image Base 0x100000 cho địa chỉ JSON được 0x44FCE70, khớp Listing. Hai địa chỉ thể hiện cùng vị trí sau khi quy đổi.

```
0x043FCE70 + 0x00100000 = 0x044FCE70
```

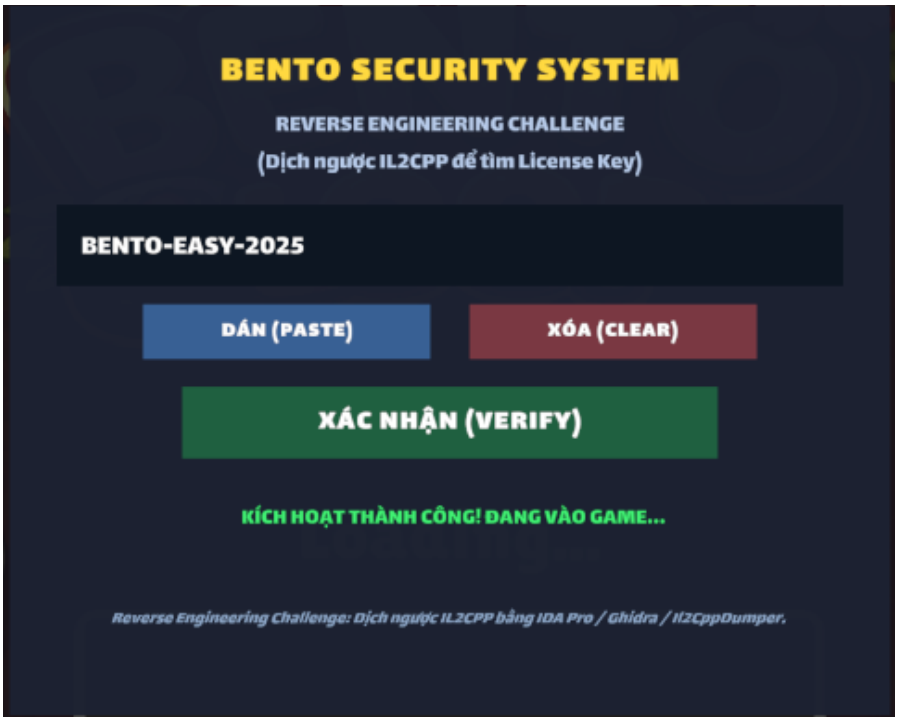
```
{
  "value": "BENTO-EASY-2025",
  "address": "0x43FCE70"
},
```

Hình 11. Giá trị và địa chỉ tương ứng trong stringliteral.json

Đối chiếu JSON xác nhận nội dung của nhãn. Vai trò của chuỗi được xác định từ phép String.Equals và nhánh trả về thành công trong VerifyLicense. BENTO_LICENSE_ACTIVATED ở vị trí khác là khóa lưu trạng thái kích hoạt, không phải key cần nhập.

6.2 Thử key trên bản APK gốc

APK gốc được mở trên thiết bị Android. Nhập BENTO-EASY-2025 vào màn hình BENTO SECURITY SYSTEM rồi bấm XÁC NHẬN (VERIFY). Ứng dụng hiển thị thông báo màu xanh KÍCH HOẠT THÀNH CÔNG! ĐANG VÀO GAME...



Hình 12. APK gốc báo kích hoạt thành công với BENTO-EASY-2025

Sau khi xác nhận, trò chơi chuyển vào Level 1. Kết quả này kiểm chứng key tìm được bằng phân tích tĩnh có hiệu lực trong APK gốc.



Hình 13. Màn chơi Level 1 sau khi vượt kiểm tra license trên APK gốc

7 Sửa VerifyLicense và xuất thư viện

7.1 Sửa hai lệnh đầu hàm

Chương trình được lưu thành bản sao libil2cpp_patched trong project Ghidra. Tại địa chỉ 0x1F7689C, hai lệnh đầu của VerifyLicense được thay bằng mov w0, #0x1 và ret qua Patch Instruction. Lệnh đầu ghi giá trị 1 vào thanh ghi trả về, lệnh sau kết thúc hàm ngay.

Địa chỉ Ghidra	Lệnh trước khi sửa	Lệnh sau khi sửa
0x1F7689C	str x30, [sp, #-0x20]!	mov w0, #0x1
0x1F768A0	stp x20, x19, [sp, #0x10]	ret

```
0x1F7689C   20 00 80 52   mov w0, #0x1
0x1F768A0   C0 03 5F D6   ret
```

Decompile: GeckoArrow.Runtime.LoadingSystem.LicenseVerifier$$VerifyLicense - (libil2...

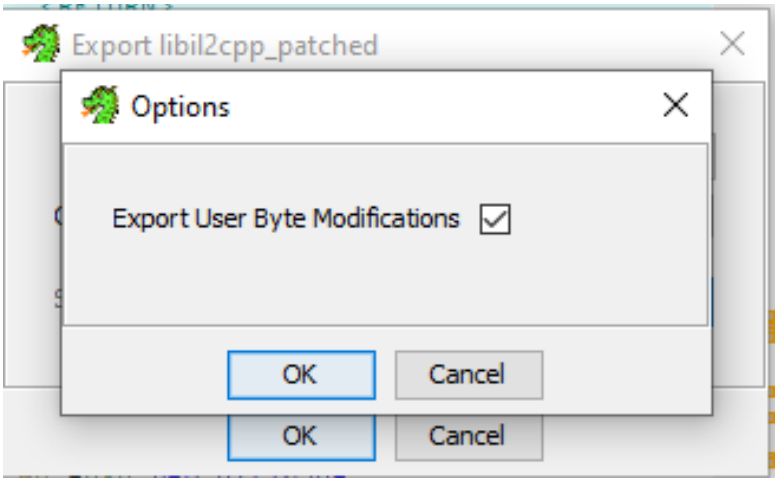
```
1
2 undefined8 GeckoArrow.Runtime.LoadingSystem.LicenseVerifier$$VerifyLicense(void)
3
4 {
5   return 1;
6 }
7
```

Hình 14. VerifyLicense sau patch hiển thị return 1 trong Decompiler

Hai lệnh mới thực hiện trước khi hàm cấp phát stack hoặc gọi hàm con, nên kết quả trả về không còn phụ thuộc các nhánh kiểm tra phía sau. Decompiler rút gọn thân hàm thành return 1. Phần lệnh cũ còn nằm trong file nhưng không được đi tới từ điểm vào hàm này.

7.2 Xuất file có chứa phần mã đã sửa

Sau khi lưu chương trình Ghidra, File → Export Program được dùng để xuất thư viện. Format được chọn là Original File, đồng thời tích Export User Byte Modifications trong Options. Tùy chọn này đưa phần byte vừa patch vào file ELF kết quả.



Hình 15. Bật Export User Byte Modifications khi xuất Original File

Export Results Summary hiển thị file đích C:\Thuchanh2\libil2cpp_patched, Format Original File và kích thước 72.828.928 byte. File xuất được đổi tên thành libil2cpp.so để thay đúng tên thư viện mà ứng dụng tải.

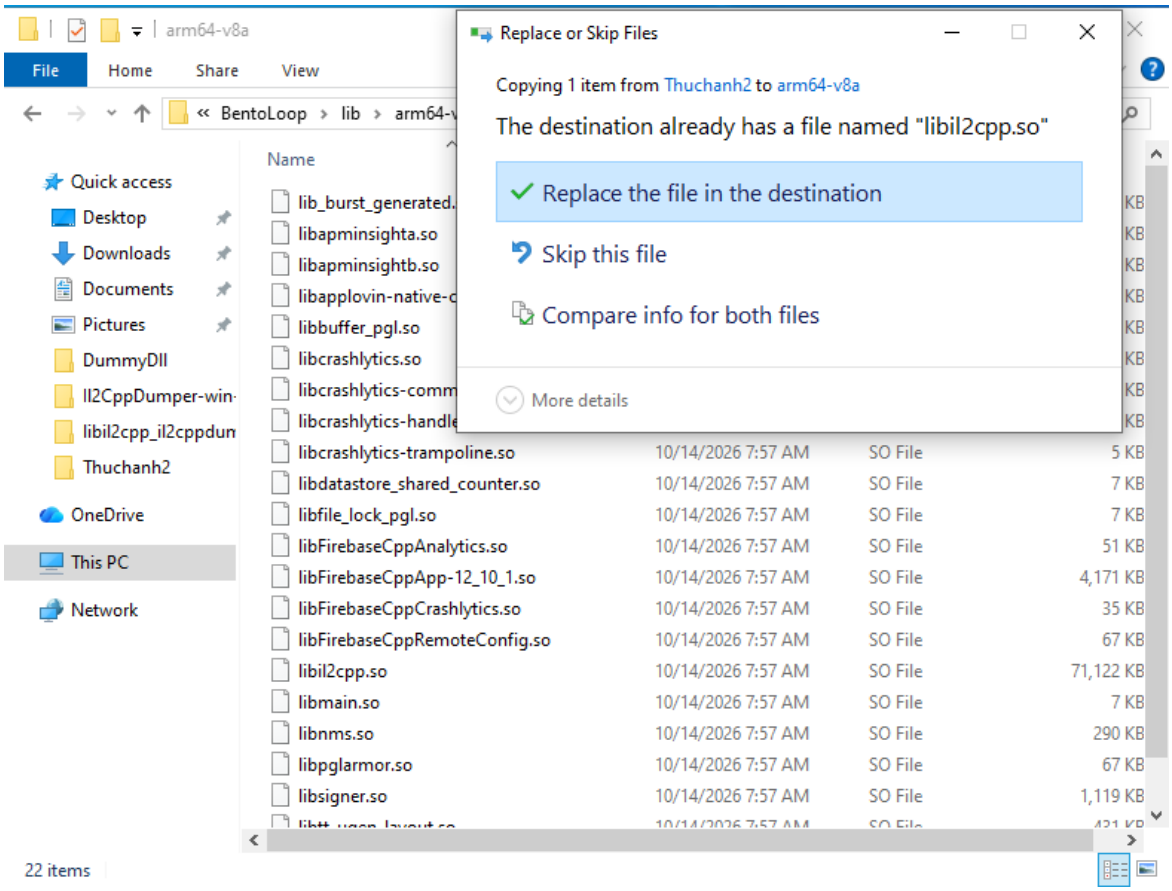
```
Additional Information
Destination file:       C:\Thuchanh2\libil2cpp_patched

Destination file Size:  72828928
Format:                 Original File
```

Hình 16. File ELF xuất ra giữ kích thước 72.828.928 byte

7.3 Thay thư viện trong thư mục đã Decompile

File libil2cpp.so đã xuất được copy vào lib\arm64-v8a của thư mục BentoLoop đã Decompile. Khi Windows báo trùng tên, chọn Replace the file in the destination. Đầu ra của bước này là thư mục APK chứa thư viện ARM64 đã patch, sẵn sàng dùng làm đầu vào Compile.



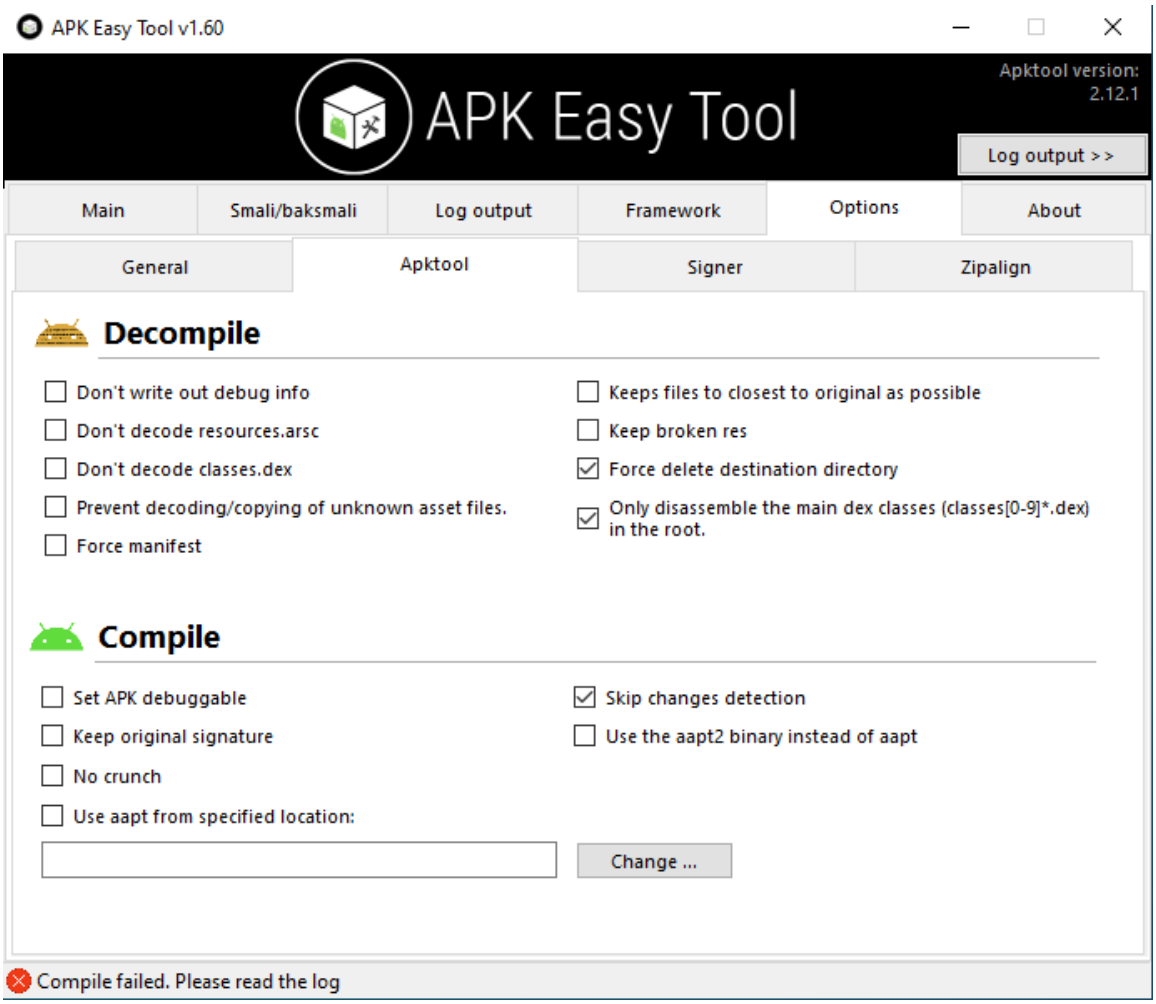
Hình 17. Thay libil2cpp.so trong thư mục arm64-v8a

8 Build APK và kiểm thử bản đã sửa

8.1 Cập nhật công cụ build

Trong APK Easy Tool, Select decompiled APK được dùng để chọn thư mục BentoLoop chứa apktool.yml, sau đó bấm Compile. Lần build đầu với Apktool 2.6.1 dừng ở manifest do không nhận thẻ property. Thư viện đã patch không phải thành phần gây ra lỗi này.

Apktool 2.12.1 được thêm vào thư mục Apktool của APK Easy Tool và chọn trong Options → General. Giao diện cũ vẫn có thể thêm tham số --use-aapt2, khiến bản mới báo Unrecognized option. Ô Use the aapt2 binary instead of aapt trong Options → Apktool được bỏ tích. Apktool 2.12.1 sử dụng AAPT2 mặc định khi build.



Hình 18. Cấu hình Apktool 2.12.1 và bỏ tùy chọn thêm use aapt2

8.2 Làm mới framework và Compile

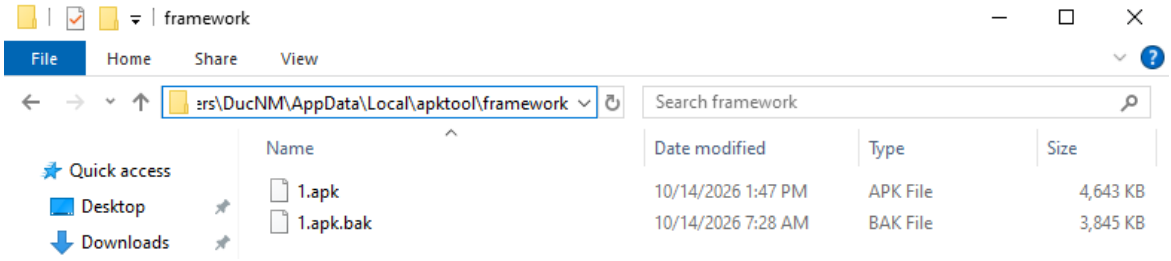
Sau khi bỏ tham số cũ, quá trình build tiếp tục nhưng báo attribute android:enableOnBackInvokedCallback not found. Log cho thấy công cụ đang dùng framework được lưu tại C:\Users\DucNM\AppData\Local\apktool\framework\1.apk. Bộ framework cũ chưa có thuộc tính này.

```
Compiling APK file... (Press ESC to cancel)
Command: java.exe -jar "C:\Thuchanh2\apk-easy-tool-1-60\Apktool\apktool_2.12.1.jar" b -f -o
"C:\Users\DucNM\Documents\APK Easy Tool\2-Recompiled APKs\BentoLoop.apk" "C:\Users\DucNM
\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop"

I: Using Apktool 2.12.1 on BentoLoop.apk with 4 threads
I: Smaling smali folder into classes.dex...
I: Smaling smali_classes2 folder into classes2.dex...
I: Smaling smali_classes10 folder into classes10.dex...
I: Building resources with aapt2...
I: Smaling smali_classes3 folder into classes3.dex...
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:85:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:126:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:127:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:128:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:129:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:130:
error: attribute android:enableOnBackInvokedCallback not found.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:165:
```

Hình 19. Lỗi thiếu enableOnBackInvokedCallback khi dùng framework cũ

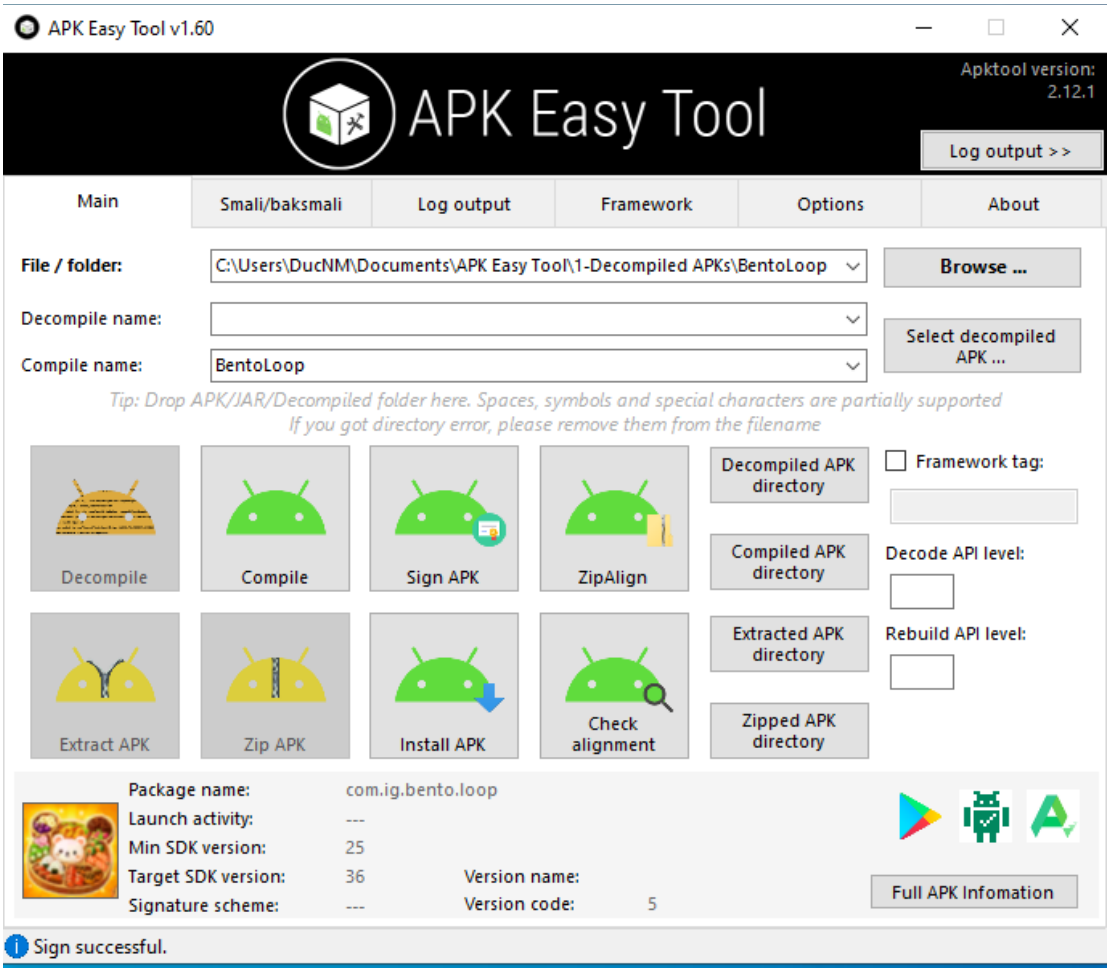
File 1.apk được đổi tên thành 1.apk.bak để giữ bản sao. Khi Compile lại, Apktool tạo file 1.apk mới từ framework đi kèm phiên bản 2.12.1. Ảnh thư mục cho thấy file mới có dung lượng 4.643 KB, còn bản sao cũ là 3.845 KB. Sau bước này, quá trình build tiến tới tạo và ký APK thành công.



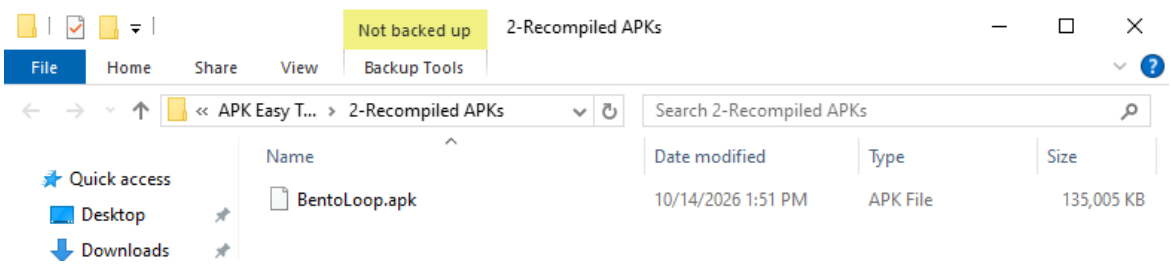
Hình 20. Framework mới được tạo bên cạnh bản sao 1.apk.bak

8.3 Lấy APK đã build và ký

APK Easy Tool hiển thị Sign successful sau quá trình build và ký. Bấm Compiled APK directory để mở thư mục 2-Recompiled APKs. File đầu ra là BentoLoop.apk, có dung lượng hiển thị 135.005 KB. Đây là APK dùng để kiểm thử bản đã sửa.



Hình 21. APK Easy Tool hoàn tất và báo Sign successful

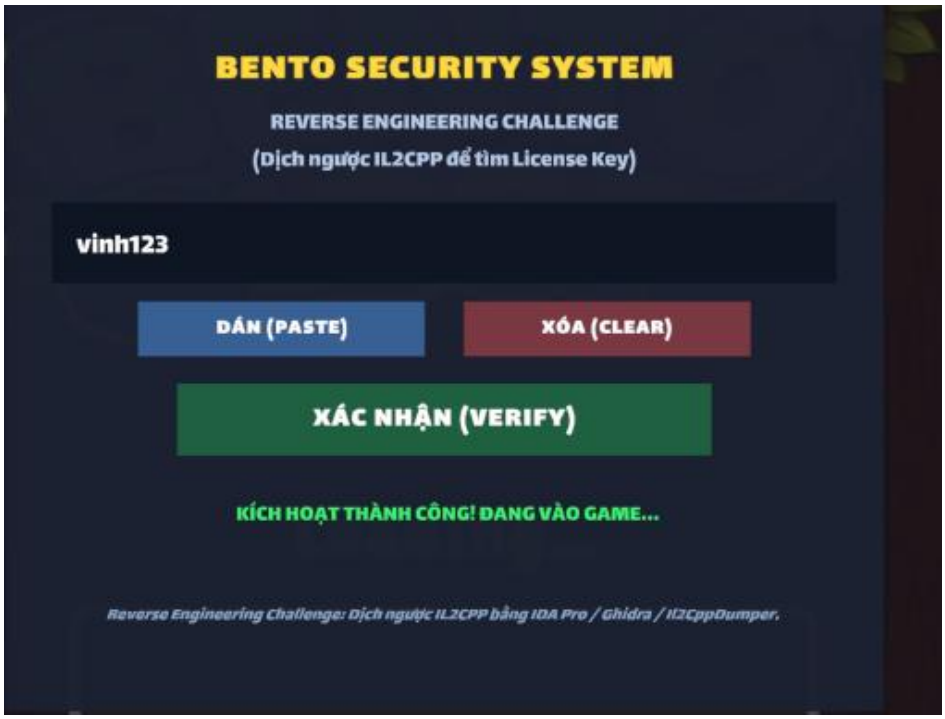


Hình 22. BentoLoop.apk trong thư mục 2 Recompiled APKs

APK được chuyển từ máy ảo sang máy thật để đưa lên thiết bị Android. Khi kiểm thử, cần dùng bản APK vừa build và bắt đầu từ trạng thái chưa kích hoạt, tránh nhầm với trạng thái được lưu từ lần nhập key đúng trước đó.

8.4 Kiểm thử với chuỗi khác key đã tìm được

Trên bản APK mới, nhập vinh123 rồi bấm XÁC NHẬN (VERIFY). Ứng dụng hiển thị KÍCH HOẠT THÀNH CÔNG! ĐANG VÀO GAME... dù chuỗi này khác BENTO-EASY-2025. Ảnh dưới được ghi nhận từ APK đã sửa, còn ảnh ở mục 6.2 thuộc APK gốc.



Hình 23. APK đã sửa báo kích hoạt thành công với chuỗi vinh123

Kết quả phù hợp với việc VerifyLicense trả về 1 ngay khi được gọi. Thử nghiệm xác nhận phần kiểm tra license tại hàm này đã được vòng tránh trên bản APK build lại. Phạm vi sửa là thư viện arm64-v8a và luồng kiểm tra license đã quan sát; thử nghiệm không đánh giá những chức năng khóa khác trong trò chơi.

Bản ứng dụng	Chuỗi thử nghiệm	Kết quả ghi nhận
APK gốc	BENTO-EASY-2025	Báo kích hoạt thành công và vào Level 1
APK đã sửa	vinh123	Báo kích hoạt thành công và đang vào game

9 Nhận xét và phương án phòng tránh

9.1 Nguyên nhân cơ chế có thể bị vòng tránh

Key BENTO-EASY-2025 được lưu dưới dạng chuỗi và so sánh trong thư viện phía client. Dữ liệu dump giúp xác định tên phương thức, từ đó lần đến phép so sánh và nội dung key. Nhánh kiểm tra mảng XOR không bảo vệ được nhánh chấp nhận chuỗi trực tiếp đang tồn tại song song.

Việc sửa hai lệnh đầu VerifyLicense cho thấy quyết định chấp nhận license phụ thuộc vào một giá trị trả về có thể thay đổi trong mã client. APK sau khi ký lại vẫn chạy được trong thử nghiệm và chấp nhận chuỗi khác key. Kết quả này cho thấy cơ chế bảo vệ đã quan sát chưa ngăn được cách can thiệp đã thực hiện.

9.2 Các biện pháp giảm rủi ro

Với ứng dụng có máy chủ, quyền sử dụng nên được xác minh theo tài khoản hoặc license trên máy chủ. Các thao tác cần quyền phải tiếp tục được kiểm tra tại nơi cung cấp dữ liệu hoặc dịch vụ. Cách này hạn chế việc sửa một màn hình hoặc một hàm client để nhận toàn bộ quyền sử dụng.

Không đưa key chung và nhánh chấp nhận thử nghiệm vào bản phát hành. Nếu cần hoạt động offline, license có thể được ký số: client giữ khóa công khai để kiểm tra, còn khóa riêng phát hành license được giữ ngoài ứng dụng. Cơ chế này giúp tránh nhúng một bí mật chung có thể đọc trực tiếp từ APK.

Kiểm tra chữ ký ứng dụng, tính toàn vẹn thư viện và phát hiện môi trường can thiệp có thể bổ sung khả năng phát hiện APK bị sửa. Làm rối mã giúp tăng chi phí phân tích nhưng không thay thế việc xác minh quyền ở phía tin cậy. Với môi trường offline, vẫn cần tính đến khả năng mã kiểm tra bị patch và lựa chọn mức bảo vệ phù hợp.

9.3 Kết luận thực hành

Bài thực hành hoàn thành việc tìm key BENTO-EASY-2025 và kiểm chứng trên APK gốc. VerifyLicense được sửa bằng hai lệnh ARM64 để trả về thành công, sau đó thư viện được xuất, thay vào thư mục đã Decompile và build lại bằng APK Easy Tool. Lỗi công cụ được xử lý bằng Apktool 2.12.1, bỏ tham số use aapt2 và làm mới framework. APK đã ký báo kích hoạt thành công với chuỗi vinh123.

Tài liệu tham khảo

Yêu cầu báo cáo và hướng dẫn APK Easy Tool được cung cấp trong bài thực hành.

Apktool 2.12.1
https://github.com/iBotPeaches/Apktool/releases/tag/v2.12.1

Tham số build của Apktool
https://apktool.org/docs/2.x/cli-parameters/

Quản lý framework trong Apktool
https://apktool.org/wiki/in-depth/frameworks/

Android Developers, phần tử property trong manifest
https://developer.android.com/guide/topics/manifest/property-element

OWASP MASVS RESILIENCE 2, bảo vệ ứng dụng trước can thiệp mã
https://mas.owasp.org/MASVS/controls/MASVS-RESILIENCE-2/

Android Developers, tổng quan Play Integrity API
https://developer.android.com/google/play/integrity/overview